

AI DevFest — Problem Statement

Tender Document Package Builder

Released at T+0 · Build time: 90 minutes

All rules in the Official Rulebook also apply.

1 Background

When an organization invites companies to compete for a tender, each bidder must submit a required set of documents such as trade license, TIN and VAT certificates, bank solvency letter, experience certificates, and technical and financial proposals. Some documents are mandatory, some are optional, and some must remain valid on the submission date. The tender says which documents are needed and in what order.

In many offices, this package is prepared manually. Staff must check the requirements, verify expiry dates, avoid duplicate files, and arrange documents in the correct order. Small mistakes like a missing, expired, duplicated, or misplaced document can make the submission incomplete or cause the bid to be rejected.

2 Your Task

Build a **web app (frontend only)** that helps office staff turn a set of PDF files into **one complete, checked and correctly ordered PDF package**, ready to submit.

3 Provided Materials

At T+0 you get `sample-pack.zip`, which contains:

- `requirements.json` — the tender details and the list of required documents;
- `documents/` — sample PDFs that the user must match to the corresponding tender requirements and include, where applicable, in the final package.

The sample pack has some real-life problems hidden in it. Your app should find them. Judges will test your app with **a different pack you have not seen**, in the same format.

Format of requirements.json

```
{
  "tender": {
    "tender_id": "T-2026-0417",
    "title": "Supply of IT Equipment",
    "procuring_entity": "Example Directorate",
    "bidder": "Example Company Ltd.",
    "submission_deadline": "2026-10-20"
  },
  "requirements": [
    { "id": "R01", "order": 1,
      "title_en": "Trade License", "title_bn": "<Bangla title>",
      "mandatory": true, "has_expiry": true },
    ...
  ]
}
```

Field	What it means
order	Where the document goes in the final package (1 = first).
mandatory	true: required; the package cannot be made without it. false: optional.
has_expiry	true: the document has an expiry date that must be checked.
submission_deadline	Last date to submit the tender, in YYYY-MM-DD format. Used to check expiry.

4 Main Tasks (must do)

- 4.1 **Load the list.** The user opens requirements.json. Your app shows the tender details and the list of required documents, sorted by order.
- 4.2 **Upload files.** The user can upload many PDF files at once. Show each file's name and number of pages. If a file is not a PDF, reject it and show a clear message. The user can remove any uploaded file.
- 4.3 **Match files.** The user matches each uploaded file to one required document. One document gets at most one file. One file goes to at most one document. The user can change or undo a match at any time.
- 4.4 **Enter expiry dates.** If a document has has_expiry = true and a file is matched to it, the user enters its expiry date.
- 4.5 **Check everything.** Show a status for every required document (see Section 5). Update the status right away after every change.
- 4.6 **Find duplicates.** If two or more uploaded files have exactly the same content (even with different names), mark them as duplicates. Do not allow them to be matched to different documents.

- 4.7 **Make the package.** Keep the *Generate* button disabled while any document has a blocking status (see Section 5), and show why. When there are no blocking problems, create one combined PDF as described in Section 6.
- 4.8 **Download.** The user downloads the package as `<tender_id>_Package.pdf`.
- 4.9 **Two languages.** The user can switch the whole app between Bangla and English. Show document names from `title_bn` or `title_en`, based on the chosen language.

5 Status Rules

Each required document shows exactly one status:

Status	When	Blocks the package?
Missing	Required document, no file matched.	Yes
Expiry date needed	<code>has_expiry = true</code> and a file is matched, but no expiry date entered.	Yes
Expired	The expiry date is <i>before</i> the submission deadline.	Yes
Not provided	Optional document, no file matched.	No
OK	File matched, and (if <code>has_expiry</code>) the expiry date is on or after the submission deadline.	No

If a document expires on the same day as the submission deadline, it is still OK. Duplicate files (task 4.6) are marked in the list of uploaded files.

6 Package Rules

The PDF your app creates must follow these rules exactly:

- 6.1 **Page 1 is a cover page**, in English. It shows: tender ID, tender title, procuring entity, bidder name, submission deadline, the date the package was made, and the list of included documents in order.
- 6.2 **The documents come after the cover**, sorted by `order`. Include all pages of each file, in their original order. Skip optional documents with no file.
- 6.3 **Every page**, including the cover, has a footer at the bottom: `<tender_id> | Page X of Y`. Y is the total number of pages in the package.
- 6.4 The footer must be easy to read and must not cover the document's content.

7 Bonus Tasks (optional)

Finish the main tasks first. Bonus tasks only get marks if the main tasks work.

- **Index page** after the cover, showing the page number where each document starts.

- **Seal or signature:** the user uploads a PNG image and places it on chosen pages.
- **Export the checklist** as Excel or CSV (document, file name, pages, expiry date, status).
- **Save and reopen** your work (for example, export/import a project file, or use browser storage).
- **Bangla text** shown correctly on the PDF cover or index page.
- **Auto-match:** suggest matches based on file names.
- **Handle bad files safely:** for damaged or password-protected PDFs, show a clear message instead of crashing.
- **AI help**, using the user's own API key (Rulebook, Section 5.5).

8 Limits and Contest Reminders

- **Frontend only.** All document processing must happen in the browser. Do not upload tender documents to any participant-controlled backend, database, or online storage service.
- Input files are PDFs only, with up to 30 files and 50 MB in total.
- The application must run in the latest Google Chrome.
- Helpful libraries (you do not have to use them): `pdf-lib` to combine PDFs and add footers; `pdf.js` to count pages and show previews.
- **Git reminder:** Commit at least **once every 30 minutes**, with at least **3 commits in total**. Each commit message must briefly state what changed and include the AI prompt used, or `Manual edit` if no AI was used.
- Your final eligible commit and matching public HTTPS deployment must be completed by **T+90**. Stop coding, committing, pushing, and changing the deployment at T+90.

9 What to Submit

Along with everything required by the Rulebook (Section 9), submit the following:

In your GitHub repository:

- `output/<tender_id>_Package.pdf` — the final package generated from the provided sample pack after resolving its problems.
- `screenshots/` — including at least one screenshot showing the document statuses.

Through the submission portal:

- your public GitHub repository URL;
- your **public HTTPS live website link**, accessible to the judges without login or special permission.

10 What Judges Will Look For

Marks follow the Rulebook (Section 11). For this problem, judges will especially check:

- Does your app show the right status (Section 5) for every document in the unseen pack?
- Does the PDF follow Section 6 exactly (order, cover page, page numbers)?
- Can an office worker with no tech skills finish the task in either language without help?